

3.3 Servidores TCP concurrentes

Programación de servicios y procesos

Curso 2025/2026

Servidores secuenciales

- ▶ Hasta ahora: servidor TCP para **un solo cliente**
- ▶ El servidor:
 - ▶ acepta una conexión
 - ▶ atiende al cliente
 - ▶ termina o espera a que cierre
- ▶ Problema:
 - ▶ mientras un cliente está conectado, **no se pueden atender otros**

¿Qué ocurre si llegan varios clientes?

- ▶ El protocolo TCP admite varias conexiones simultáneas
- ▶ `ServerSocket.accept()`
 - ▶ acepta una conexión cada vez
 - ▶ es bloqueante
- ▶ Si un servidor está atendiendo a un cliente, los demás deben esperar

Idea clave: separar responsabilidades

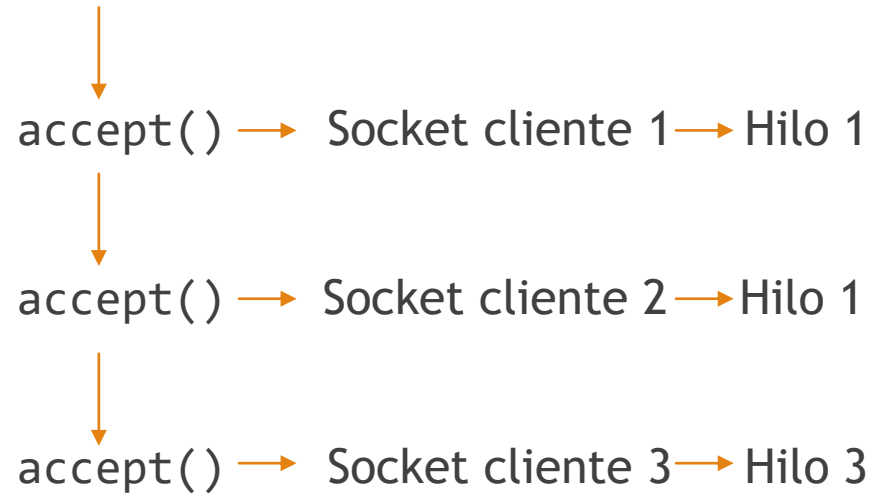
- ▶ Dos tareas distintas:
 - ▶ escuchar nuevas conexiones
 - ▶ comunicarse con el cliente
- ▶ Cada cliente tiene su propio flujo de ejecución

Solución con hilos

- ▶ El servidor
 - ▶ Ejecuta `accept()` en un bucle
- ▶ Cada vez que llega un cliente
 - ▶ se crea un `Socket`
 - ▶ se lanza un hilo para atenderlo
- ▶ El hilo:
 - ▶ gestiona streams
 - ▶ lee y escribe
 - ▶ finaliza cuando el cliente cierra

Esquema de funcionamiento

Hilo principal (servidor)



Implementación básica en Java

- ▶ Servidor:
 - ▶ Bucle infinito con accept
 - ▶ Crea un Runnable por cliente
 - ▶ Lanza un hilo
- ▶ El runnable:
 - ▶ Mismo código de comunicación que si hubiera con un solo cliente
- ▶ El hilo termina cuando
 - ▶ read() devuelve -1 o readline devuelve null → se cierran streams y socket
- ▶ El servidor sigue escuchando

Actividad propuesta

- ▶ AP4. Servidor Eco concurrente (modo bytes) - ejercicio 1

¿Cómo limitar el número de clientes concurrentes?

- ▶ Cada cliente atendido consume CPU (tiempo de procesador)
- ▶ ¿Cómo limitarlos? → Limitando el número de hilos

¿Cómo limitar el número de hilos?

```
ExecutorService pool = Executors.newFixedThreadPool(m);
```

- ▶ m es el número máximo de clientes concurrentes
- ▶ Cada cliente se atiende en un hilo del pool
- ▶ Si llegan más de m clientes:
 - ▶ no se rechazan
 - ▶ quedan en espera en la cola del executor
- ▶ Cuando un cliente termina:
 - ▶ se libera un hilo
 - ▶ el siguiente cliente en espera comienza su atención

Actividad propuesta

- ▶ AP4. Servidor Eco concurrente (modo bytes) - ejercicio 2

¿Cómo limitar la cola?

- ▶ Un cliente puede abrir ~65.000 puertos locales
- ▶ Muchos clientes → muchísimas conexiones simultáneas
- ▶ Cada conexión consume:
 - ▶ memoria (buffers, objetos Socket)
 - ▶ descriptores de fichero
 - ▶ tiempo de CPU
- ▶ Mucho antes de llegar al máximo:
 - ▶ la JVM se queda sin memoria
 - ▶ o el SO sin recursos

Opción 1 - cola acotada

```
ExecutorService pool =  
    new ThreadPoolExecutor(  
        m, m,                               // corePoolSize = m, maximumPoolSize = m  
        OL, TimeUnit.MILLISECONDS,         // keepAliveTime = 0 ms  
        new ArrayBlockingQueue<>(N)         // cola  
    );
```

- ▶ $m \rightarrow$ clientes activos
- ▶ $N \rightarrow$ máximo de clientes en espera
- ▶ Si se supera N : la tarea se rechaza (`RejectedExecutionException`)

Opción 2 - rechazo de conexión

- ▶ Antes de enviar al pool

```
if (clientesEnEspera.get() > MAX_ESPERA) {  
    socket.close();  
    System.out.println("Cliente rechazado");  
    continue;  
}
```

Ventajas de limitar la ejecución y la cola

- ▶ Limitar el número de hilos: asegura dar un buen servicio
 - ▶ Solo se atienden las peticiones para las que se garantiza atención inmediata
- ▶ Limitar la cola: protege contra cargas excesivas
 - ▶ Por ejemplo, si un cliente abre más conexiones de las esperadas

¿Limitar la cola protege contra un DoS?

- ▶ Recordemos: DoS - *denial of service* - ataque consistente en solicitar millones de conexiones
- ▶ Respuesta: **NO**
 - ▶ El DoS afectaría a la pila TCP/IP dejándola colapsada
 - ▶ Estos ataques tienen que prevenirse en niveles inferiores
- ▶ Ahora bien, si limitamos la cola la máquina virtual Java seguiría viva
 - ▶ El efecto de un DoS sería que el servidor pierde la conexión al exterior
 - ▶ El servidor no ha protegido al sistema, pero se ha protegido a sí mismo